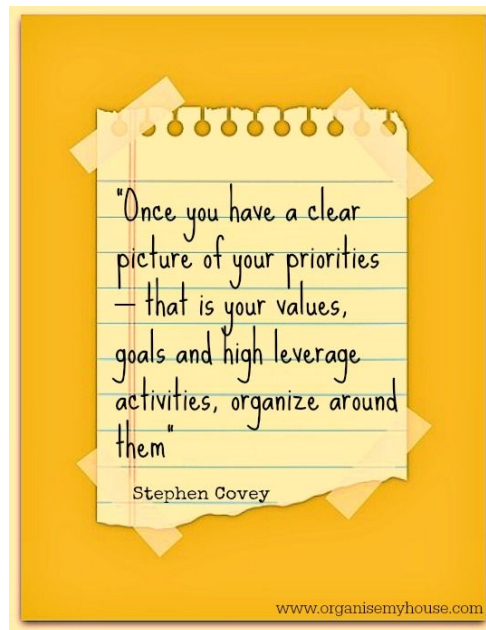


Recursion 3



Content for Today

* Quizzes

01. Generate K^+ symbol
02. Generate no. using 1 & 2
03. count of subsets with $sum = K$

```
int magicfun( int N) {      N=7
    if ( N == 0)
        return 0;
    else
        return magicfun(N/2) * 10 + (N % 2);
}
```

$$11 * 10 + 7 \% 2 = 110 + 1 = 111$$

⑪

```
int magicfun( int N) {      N=3
    if ( N == 0)
        return 0;
    else
        return magicfun(N/2) * 10 + (N % 2);
}
```

$$1 * 10 + 3 \% 2 = 10 + 1 = 11$$

①

```
int magicfun( int N) {      N=1
    if ( N == 0)
        return 0;
    else
        return magicfun(N/2) * 10 + (N % 2);
}
```

$$0 * 10 + (1 \% 2) =$$

0

```
int magicfun( int N) {      N=0
    if ( N == 0)
        return 0;
    else
        return magicfun(N/2) * 10 + (N % 2);
}
```

TC: $O(\log N)$

SC: $O(\log N)$

Quiz 2

7:13 → 7:15 AM

SCROLL , 0

```
1. void fun(char s[], int x) {
2.     System.out.println(s);
3.     char temp;
4.     if(x < s.length/2) {
5.         temp=s[x];
6.         s[x] = s[s.length-x-1];
7.         s[s.length-x-1]=temp;
8.         → fun(s, x+1);
9.     }
10. }
```

Run for fun("SCROLL", 0);

LCROLS

Tc: $O(n)$

Sc: $O(n)$

LCROLS , 1

```
1. void fun(char s[], int x) {
2.     System.out.println(s);
3.     char temp;
4.     if(x < s.length/2) {
5.         temp=s[x];
6.         s[x] = s[s.length-x-1];
7.         s[s.length-x-1]=temp;
8.         → fun(s, x+1);
9.     }
10. }
```

Run for fun("SCROLL", 0);

LLROCS

SCROLL

LCROLS

LLROCS

LLORCS

LLROCS 2

```
1. void fun(char s[], int x) {
2.     System.out.println(s);
3.     char temp;
4.     if(x < s.length/2) {
5.         temp=s[x];
6.         s[x] = s[s.length-x-1];
7.         s[s.length-x-1]=temp;
8.         → fun(s, x+1);
9.     }
10. }
```

Run for fun("SCROLL", 0);

LLORCS



```

1. void fun(char s[], int x) {
2.     System.out.println(s);
3.     char temp;
4.     if(x < s.length/2) {
5.         temp=s[x];
6.         s[x] = s[s.length-x-1];
7.         s[s.length-x-1]=temp;
8.         fun1(s, x+1);
9.     }
10. }

```

Run for fun("SCROLL", 0);

01 K⁺ symbol

Given N, K . Find the K^{th} ele in N^{th} row

$$0 \rightarrow \{0, 1\}$$
$$1 \rightarrow \{1, 0\}$$
[illegible]

row	col		
n	k		Ans ↓
2	1	→	1
3	2	→	1
4	5	→	0

AL<I> recur (int N , int K)

if (n==0){

AL<I> base = new AL<>();

base.add(0);

return base;

AL<I> ans = recur (N-1, K)

AL<I> ans = new AL<>();

for (i=0 ; i<ans.size(); i++) {

int val = ans.get(i);

if (val==0){

ans.add(0);

ans.add(1);

else {

ans.add(1)

ans.add(0)

return ans;

```
main() {
```

```
    AL<I> ans = recur(n, k)
```

```
    return ans.get(k);
```

```
}
```

TC: $N \rightarrow (N-1) \rightarrow (N-2) \rightarrow (N-3) \dots \rightarrow 1 \rightarrow 0$

$2^n + 2^{n-1} + 2^{n-2} + 2^{n-3} + \dots + 2^1 + 2^0$

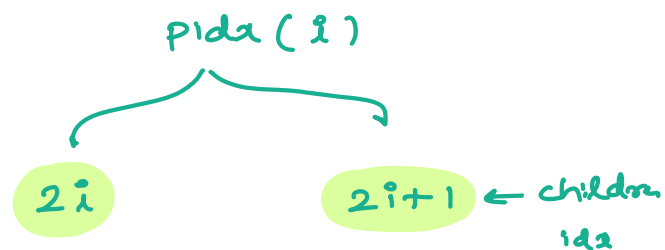
TC: $O(2^n)$

SC: $O(2^n)$

* Optimised solution

Observation $\rightarrow$ Every no. is going to generate two child no.

ida	children ida
0	0 & 1
2	4 & 5
1	2 & 3

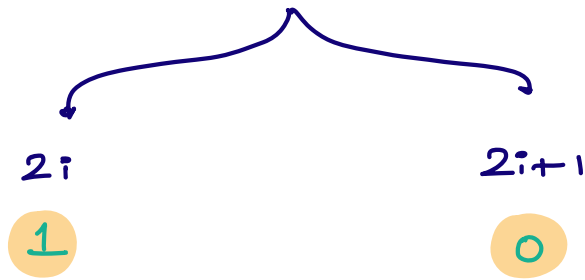


cida
(i)

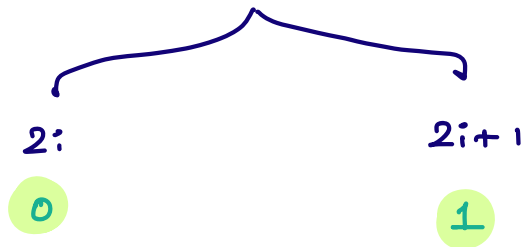
pida
 $i/2$

Obs 2

parent value = 1



parent value = 0



Conclusion → even idx child has the same value as parent

→ odd idx child has different value than parent

```
int fun(n, k)
{
    if (n == 0 || k == 0) return 0;

    int p = fun(n-1, k/2);

    if (k % 2 == 0) return p;
    else return 1-p;
}
```

TC: $O(\log k)$
SC: $O(\log k)$

02. Given N , print all N digits numbers formed by only 1 & 2 in increasing order of numbers.

$N=2$

<u>1</u>	<u>1</u>
<u>1</u>	<u>2</u>
<u>2</u>	<u>1</u>
<u>2</u>	<u>2</u>

$N=3$

<u>1</u>	<u>1</u>	<u>1</u>
<u>1</u>	<u>1</u>	<u>2</u>
<u>1</u>	<u>2</u>	<u>1</u>
<u>1</u>	<u>2</u>	<u>2</u>
<u>2</u>	<u>1</u>	<u>1</u>
<u>2</u>	<u>1</u>	<u>2</u>
<u>2</u>	<u>2</u>	<u>1</u>
<u>2</u>	<u>2</u>	<u>2</u>

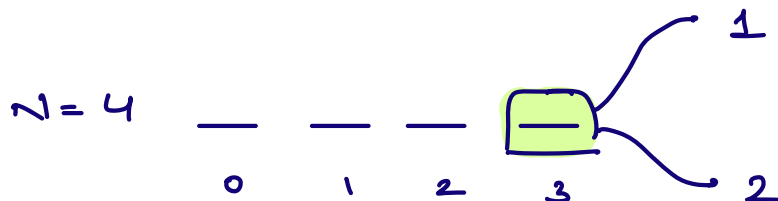
Idea

01. Use binary representation of ele 0 to $2^n - 1$

— First generate binary of no. one by one

→ While representing binary, if rem comes out to be zero, add 1 else, if remainder comes out to be 1, add 2

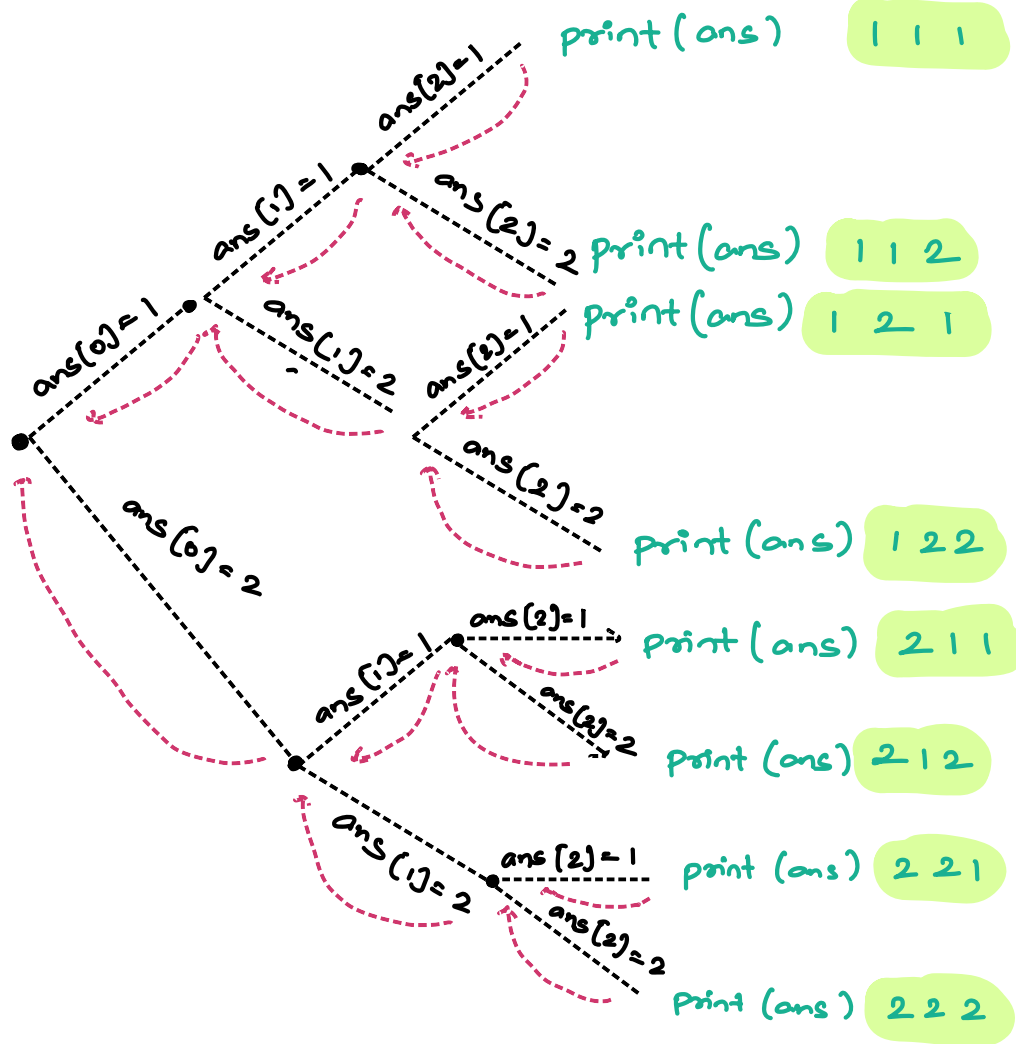
02 Use backtracking



$$\underline{N=3}$$

$$ans = \begin{bmatrix} 2 & 2 & 1 \\ 0 & 1 & 2 \end{bmatrix}$$

$i = 0 \quad 1 \quad 2 \quad 3$



```
void printall (int [] ans, i, N)
```

```
    if (i == N) {
```

```
        print(ans);
```

```
        return;
```

```
    // at i+1 idx
```

```
    ans[i] = 1;
```

```
    printall (ans, i+1, N)
```

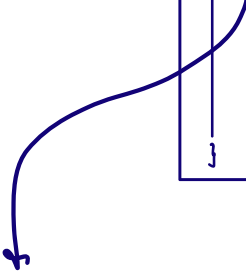
```
    ans[i] = 2;
```

```
    printall (ans, i+1, N);
```

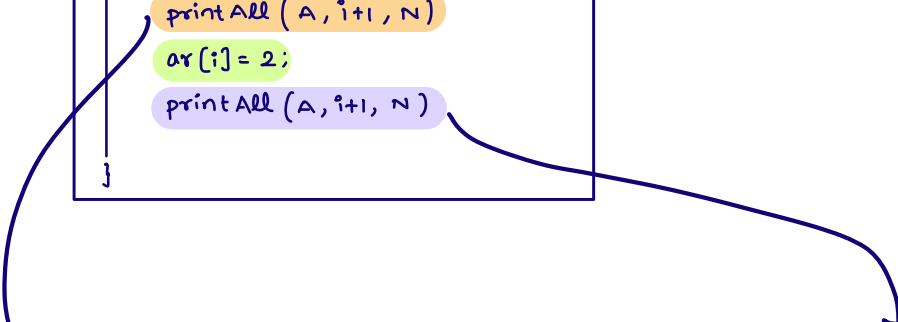
Tc: $O(2^n)$

Sc: $O(n)$

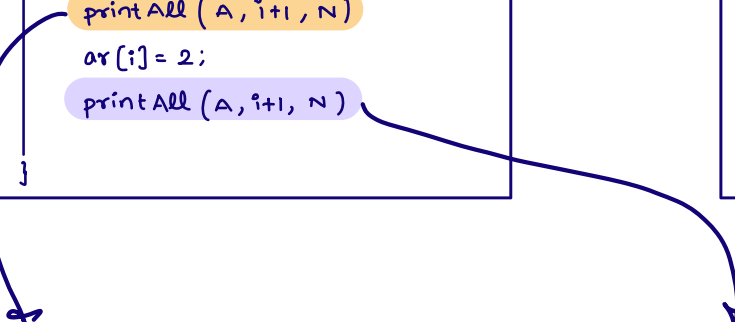
```
void printAll(int CJA, i=0, N=3)
{
    if (i == N) { print(CJA) return; }
    ar[i] = 1;
    printAll(A, i+1, N)
    ar[i] = 2;
    printAll(A, i+1, N)
}
```



```
void printAll(int CJA, i=1, N=3)
{
    if (i == N) { print(CJA) return; }
    ar[i] = 1;
    printAll(A, i+1, N)
    ar[i] = 2;
    printAll(A, i+1, N)
}
```



```
void printAll(int CJA, i=2, N=3)
{
    if (i == N) { print(CJA) return; }
    ar[i] = 1;
    printAll(A, i+1, N)
    ar[i] = 2;
    printAll(A, i+1, N)
}
```



```
void printAll(int CJA, i=2, N=3)
{
    if (i == N) { print(CJA) return; }
    ar[i] = 1;
    printAll(A, i+1, N)
    ar[i] = 2;
    printAll(A, i+1, N)
}
```

```
void printAll(int CJA, i=3, N=3)
{
    if (i == N) { print(CJA) return; }
    ar[i] = 1;
    printAll(A, i+1, N)
    ar[i] = 2;
    printAll(A, i+1, N)
}
```

```
void printAll(int CJA, i=3, N=3)
{
    if (i == N) { print(CJA) return; }
    ar[i] = 1;
    printAll(A, i+1, N)
    ar[i] = 2;
    printAll(A, i+1, N)
}
```

03 Given $ar[N]$ elements, find count of subsets with $sum = K$

$$ar[] = \{5, 7, 2\} \quad K = 7$$

$$\text{subsets} \Rightarrow \{(5, 2), (7)\} \quad \underline{\underline{\text{count} = 2}}$$

subset $\Rightarrow$ delete zero or more ele from arr to generate subset

$$\begin{array}{lcl} \{ \} & = & 0 \\ 5 & = & 5 \\ 7 & = & 7 \\ 2 & = & 2 \\ 5, 2 & = & 7 \\ 5, 7 & = & 12 \\ 7, 2 & = & 9 \\ 5, 7, 2 & = & 14 \end{array} \quad \left. \vphantom{\begin{array}{lcl} \{ \} \\ 5 \\ 7 \\ 2 \\ 5, 2 \\ 5, 7 \\ 7, 2 \\ 5, 7, 2 \end{array}} \right\} \quad \underline{\underline{\text{count} = 2}}$$

Idea $\rightarrow$ Generate sum of subset using backtracking & check if it is K or not

$$ar[] = \{5 \quad 2 \quad 7\} \quad \checkmark \quad \underline{\underline{K = 7}}$$

✓
x

int countsubsum (idx, ^{size of array} n, sum, k, ar)

if (idx == n){

if (sum == k) return 1;

else return 0;

}

sum = sum + ar[idx];

int Ca = countsubsum (idx + 1, n, sum, k, ar);

sum = sum - ar[idx];

int Cb = countsubsum (idx + 1, n, sum, k, ar);

return Ca + Cb;

Tc: $O(2^n)$

Sc: $O(n)$

}